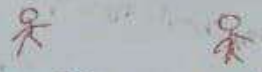
Inheritance

Scenario 1 — Company Workers



He was Treated differently

But manager can do more specialist tasks

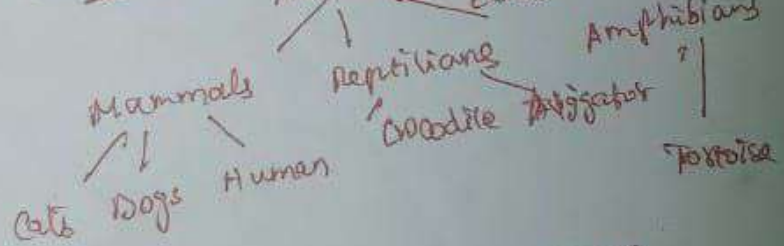
Employee do some common tasks

Both are Employees

{Manager is an specialized form of employee}

"is a Relationship"

Scenario 2 — Animal → Common class



But Basically all are animals

Each one is specialized form

Here is the process of concept called Inheritance

Subclass Defined

Subclass is class that inherits from superclass. It inherits non private fields ~~Instance variables~~ & methods from Superclass

```
class Animal
{
public void run(){ System.out.println("Animal run");
                 }
}

class Cat extends Animal
{ public void run(){
        System.out.println("Meow");
        }
}
```

extends is a key word used to extend the ~~subclass~~ superclass into subclass

Subclass → more specialised form of superclass

Private → cannot be inherited into subclass

Most general → superclass

Most specialised → subclass

Records → Cannot be extended and also cannot be used as superclass

Overriding method

```
class Emp {
    double salary;
    public void raiseSalary() {
        salary += salary * 0.02;
    }
}

class Manager extends Emp
    public void
```

}

overridding

overriding methods

```
class Employee {
  double salary;
  Employee()
  {
  }
  public void raiseSalary() {
  { salary += salary * 0.02; }
  }
}
```

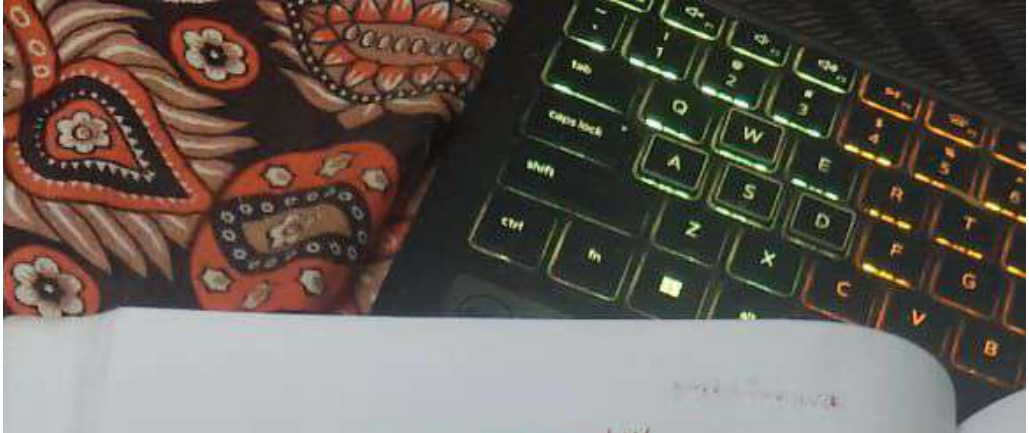
2% bonus

```
class Manager extends Employee
{
  public void raiseSalary()
  { salary += salary * 0.05; }
  }
}
```

5% bonus

Employee & manager 'P':

Redefining methods of superclass in subclass is called method overriding



How to call superclass method in subclass

```
class Employee
    private int id;
    private double salary;
    Employee(int id, double salary)
    { this.id=id;
      this.salary=salary;
    }
    public double getSalary()
    { return this.salary;
    }
}
class Manager extends Employee
{   public double getSalary()
    {
    }
}
```

* Here we need to access the salary of Employee

But it is private

So

* salary getSalary()

But method call itself more infinite calls

[StackOverflowError]

* super keyword

public double getSalary()

double ES = super.getSalary();
return S + bonus

}

{ super keyword is used to call the superclass method in subclass }

super keyword is not like this

this → refers to the current class

super → invokes or directs compiler to invoke superclass method //

Sub class Constructors

```
class Emp
{  private int id;
   private double salary;
   Emp(int id, double salary)
   {  this.id = id;
      this.salary = salary;
   }  public double getSalary()
      {  return salary
      }
}
class Manager extends Emp
{  Public Manager(int id, double salary)
   {
      --> super();
   }
}
```

-> Invisible

It won't compile

Because when you give Constructor super() keyword It checks whether superclass has default Constructor

super(); --> No default constructor in superclass

super() → calls superclass constructor } invokes

- class Manager extends Emp
  private double bonus;
{ public Manager (int id, double salary)
  { super(id, salary);
  }
  public double getSalary()
  { return super.getSalary() + bonus
  }
}

Manager cannot access private fields of Employee so we must initialize through superclass constructor

Both this & super cannot be used at one constructor this() & super()

Because They need to place at first line;

Employee[] staff = new Employee[3];

Manager boss = new Manager ("Gabi", 50000);

staff[0] = boss;
staff[1] = new Employee ("Harry", 50000);
staff[2] = new Employee ("Alaptin Laches", 60000);

for

```
for(Employee e : staff)
{
    S.o.p(e.getSalary());
}
```

Note Manager alone → can get bonus

declared type of e → Employee

actual object

Manager or Employee

Virtual machine cares only about actual object

object Mandate refer to multiple types → polymorphism

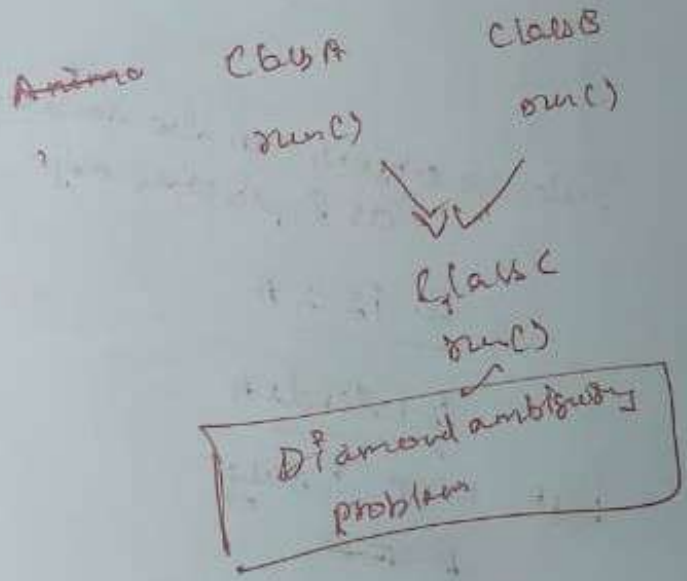
appropriate method at runtime → Dynamic Binding

Inheritance hierarchies

→ inheritance hierarchy

path from particular class to ancestor is called inheritance chain.

Java Does not support multiple inheritance



Polymorphism

Poly — Many

morph — forms (~~...~~ → foreign word)

. is-a relationship < inheritance >

class A{ → superclass

}

class B extends (A)
{ → subclass

}

subclass object can also declared as superclass obj...

B is a A

B extends A

But Here I say
~~A does~~

Is A extends B ?

No

so A is a B (False)

Superclass object is not = subclass object

This is main clarity we need to know before understanding polymorphism

class Employee{

}

Every employee are Employee

class manager extends Employee

{

}

Poly morphism → {one pillar of object oriented programming}

Poly → many
morph → forms

Before

Example

Class Animal{

}

Class cat extends Animal

{

}

is-a relationship

Cat is a Animal

Because Cat extend Animal

Subclass object can also be Superclass object

Animal is = Cat ✗

~~Default~~ d

Since Animal does not Extend Cat

Superclass object cannot be a Subclass object

Substitution principle

You can use Subclass object in area of ~~object~~ be Superclass object

Subclass object is also Superclass object

Object_variables

class Employee {

}

class Manager extends Employee {

}

Employee e ; → employee reference variable

e = new Employee ();

e = new Manager ();

Both possible

~~Class~~

Class Employee {
private double salary;
private ~~double~~ int id;
}

Class Manager extends Employee
{

}

Manager cannot access private Employee things

e = new Manager()

object is manager's

~~It can't access private things~~

```
class Employee {
    double salary;
    int id;
}
```

3

```
class Manager extends Employee {
double Bonus;
}
```

3

```
Employee e = new Employee();
e = new Manager();
```

now fine

```
Employee[] staff = new Employee[3];
staff[0] = new Manager();
staff[1] = new Employee();
staff[2] = new Employee();
```

Superclass reference → Subclass object

Array of Employee references

staff[0].bonus ✗
staff[1].bonus ✗ → Employee class

It does not contain field bonus

staff[0] → manager object
→ bonus field is not access

{ Because fields/methods specific to sub class cannot be accessed by superclass reference }

~~class Emp~~{

class Animal{
Public void sound()
{System.out.println("make sound");
}}

class Cat extends Animal
{ Public void sound()
{ system.out.println("Meow");
}}

↓ superclass reference ↓ subclass object
Animal animal = new Cat();

animal.sound();
prints Meow

Both methods are common
It prints according to object type (Dynamic binding)

Dynamic Binding

References are that

~~Abby~~ method calls depends on actual object it specifies

↓

only if superclass contains method with method name

{ specific ones can be called by only subclass reference }

Manager m = an Employee

But we cannot do this

~~Employee~~ Manager m = new Employee();

✗

Won't work

Manager

~~Employee~~ e[] = new Manager[5];

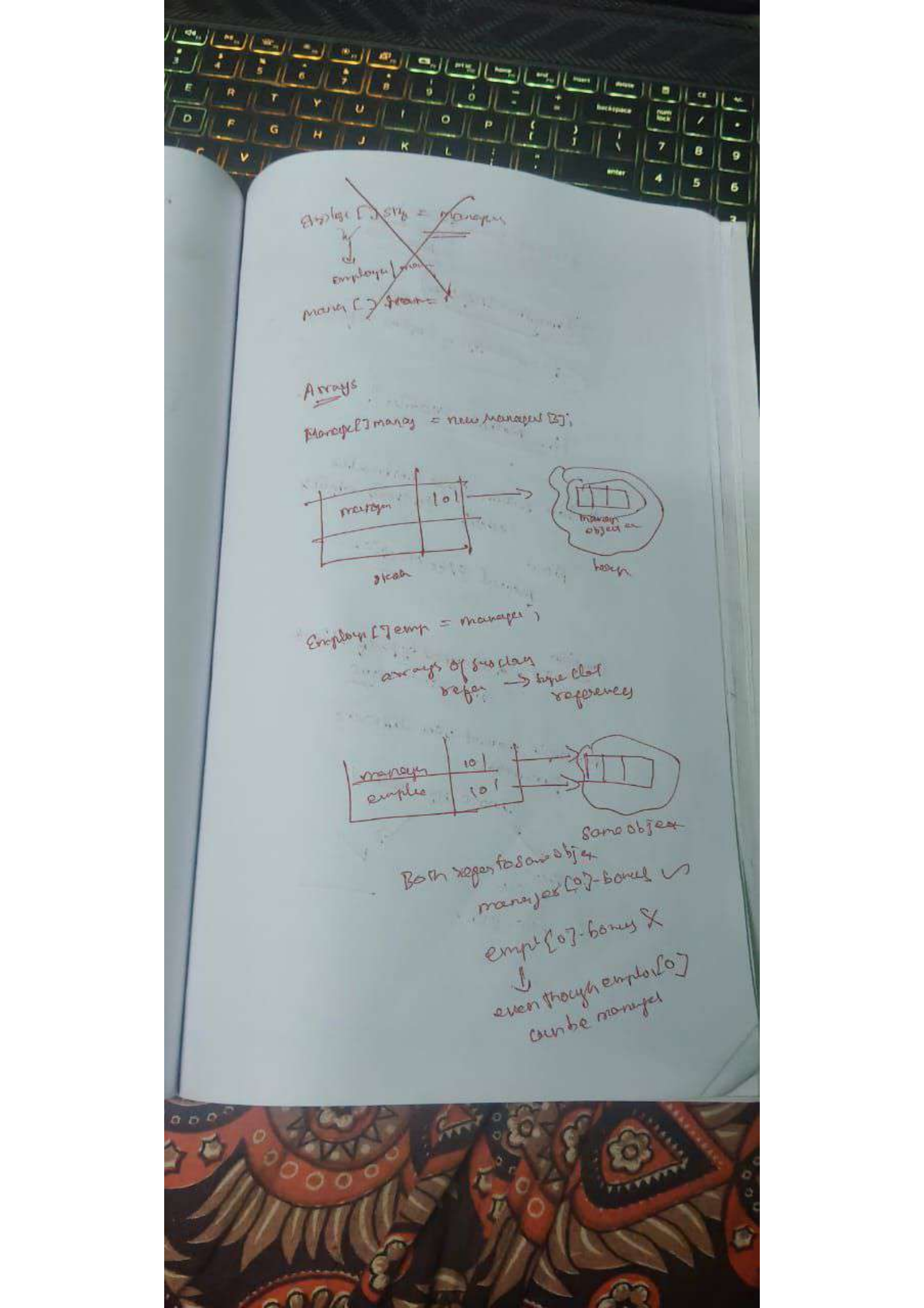
↓

manage references

(heap memory)

Employee reference

ArrayStore exception


Arrays
Manager[] manag = new Manager[3];
manager
101
stack
heap
Employ[] emp = manager;
array of subclass refer → type clash reference
manager
employee
101
101
same object
Both refers to same object
manager[0].bonus ✓
emp[0].bonus ✗
even though emp[0] can be manager

Manager[] x ← does not store angle references

[ArrayStore Exception]

To avoid storing Employee references

How Methods Executes ?

In last section How Constructor executes To create objects

Now
method Execution

Compiler → # declared type of object and method name

Multiple method with different Parameters

choose right method

If there is no Error

Compile-error

① Choosing

compiler encounters called f → [E]

accessible methods

private methods ✗

✓ method name

Not method name

Compile - error

② compiler → determines parameters or type of arguments passed

methods → must be unique method ↓

bar many parameters

Not

Compile error

↳ (method chosen)

overloaded

Now compiler knows name & parameter